Worked for 54s

The key limitation is this:

From one monocular camera frame, the target's absolute distance and relative altitude are generally not observable. The tracker gives you a bearing—where the target is—but not how far away it is. Therefore, a maximum-speed trajectory requires a range-estimation stage or an additional depth sensor.

Recommended control architecture

```
Camera tracker
      ↓ pixel center / bounding box
Bearing + range estimator
      ↓ relative target position and velocity
Online trajectory planner
      ↓ desired velocity / acceleration / yaw
RC command controller
      ↓ roll, pitch, yaw, throttle
Betaflight / flight controller
```

Use three nested loops:

Layer	Typical rate	Purpose
Visual tracker	30–60 Hz	Target pixel location and bounding box
Guidance / trajectory planner	20–50 Hz	Desired 3D velocity and acceleration
Flight controller	200–1000 Hz	Stabilization and motor control

1. Convert the target pixel into a direction

Given camera calibration matrix:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

and target center (u, v):

$$\mathbf{r}_c = \frac{K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}}{\left\| K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \right\|}$$

This gives a unit ray in the camera frame.

Transform it into the world or navigation frame:

$$\mathbf{r}_w = R_{wb} R_{bc} \mathbf{r}_c$$

where:

- R_{bc}: camera-to-body rotation
- R_{wb}: body-to-world rotation from the IMU or flight controller

You now know the direction to the target, but still not its distance.

---

2. Estimate distance and altitude

Best solution: add a range source

For aggressive maximum-speed flight, use one of:

- Stereo camera
- LiDAR
- Laser rangefinder
- Depth camera
- Radar
- Known digital terrain elevation combined with GPS/VIO

This directly gives:

$$\mathbf{p}_{target} = \mathbf{p}_{drone} + d\mathbf{r}_w$$

Then relative altitude is:

$$h_{relative} = z_{drone} - z_{target}$$

Ground-plane intersection

When the target is on flat ground and you know the drone altitude above that ground, intersect the camera ray with the ground plane.

For ground height z_g:

$$\lambda = \frac{z_g - C_z}{r_{w,z}}$$

$$\mathbf{p}_{target} = \mathbf{C} + \lambda \mathbf{r}_w$$

This works poorly when:

- The ray is close to parallel to the ground
- Ground elevation is unknown
- The target is on a roof, vehicle, hill or elevated structure
- Altitude estimates have significant error

Known target size

When the real target width W is known and its image width is w_{px}:

$$d \approx \frac{f_x W}{w_{px}}$$

This is approximate because target orientation changes its apparent width.

Monocular triangulation using drone motion

When you do not know target size, make a short lateral motion while keeping the target visible:

```
Position 1 ——— camera ray 1 ———\
                                Target
Position 2 ——— camera ray 2 ———/
```

Using the drone poses from VIO/GPS and the corresponding target bearings, triangulate the target position.

For OpenCV:

Python Run

```
target_h = cv2.triangulatePoints(
    projection_matrix_1,
    projection_matrix_2,
    pixel_1,
    pixel_2,
)

target_xyz = target_h[:3] / target_h[3]
```

The lateral movement must be large enough to create useful parallax. Moving directly toward the target produces poor depth observability.

For maximum-speed operation, maintain an EKF with a target state such as:

$$\mathbf{x} = \begin{bmatrix} p_x & p_y & p_z & v_x & v_y & v_z \end{bmatrix}^T$$

The tracker supplies bearing measurements, while VIO/GPS and IMU supply the camera poses.

---

3. Do not initially command maximum speed

Use a state machine:

```
SEARCH
  ↓
TRACK AND CENTER
  ↓
ESTIMATE RANGE
  ↓
ACCELERATE
  ↓
HIGH-SPEED APPROACH
  ↓
BRAKE
  ↓
HOVER / ORBIT AT SAFE DISTANCE
```

A good strategy is:

1. Center the target.
2. Perform a small lateral motion for triangulation.
3. Wait until target distance uncertainty is sufficiently low.
4. Accelerate toward the target.
5. Replan continuously.
6. Brake before reaching a safe stand-off radius.

Without a reliable range estimate, use only a conservative bearing-following speed.

---

4. Calculate the maximum safe approach speed

Let:

- d: current target distance
- d_{safe}: desired final stand-off distance
- a_{brake}: reliable braking acceleration
- $v_{\max}$: vehicle speed limit

The maximum speed from braking distance is:

$$v_{\text{stop}} = \sqrt{2a_{\text{brake}} \max(d - d_{\text{safe}}, 0)}$$

Then:

$$v_{\text{command}} = \min(v_{\max}, v_{\text{stop}})$$

This creates natural acceleration at long range and smooth deceleration near the target.

Add a safety margin for estimation and communication latency:

$$d_{\text{margin}} = vT_{\text{latency}} + \frac{v^2}{2a_{\text{brake}}} + k_\sigma \sigma_d$$

where:

- T_{latency}: camera, tracker, network and controller latency
- σ_d: estimated range uncertainty
- k_σ: normally 2–3

Only accelerate when:

$$d - d_{\text{safe}} > d_{\text{margin}}$$

A practical speed limit is therefore:

$$v_{\text{allowed}} = \min\left(v_{\max}, v_{\text{stop}}, v_{\text{uncertainty}}, v_{\text{tracking}}\right)$$

---

5. Generate a smooth trajectory

Do not send discontinuous joystick commands. Limit velocity, acceleration and jerk:

$$\|\mathbf{v}\| \le v_{\max}$$

$$\|\mathbf{a}\| \le a_{\max}$$

$$\|\mathbf{j}\| = \left\|\frac{d\mathbf{a}}{dt}\right\| \le j_{\max}$$

A simple online controller is:

$$\mathbf{v}_{desired} = v_{\text{allowed}} \frac{\mathbf{p}_{target} - \mathbf{p}_{drone}}{\|\mathbf{p}_{target} - \mathbf{p}_{drone}\|} + \mathbf{v}_{target}$$

Then calculate acceleration:

$$\mathbf{a}_{raw} = \frac{\mathbf{v}_{desired} - \mathbf{v}_{drone}}{\tau_v}$$

Limit it:

$$\mathbf{a}_{limited} = \text{clipNorm}(\mathbf{a}_{raw}, a_{\max})$$

Then apply a jerk limit:

$$\mathbf{a}_{cmd} = \mathbf{a}_{previous} + \text{clipNorm}\left(\mathbf{a}_{limited} - \mathbf{a}_{previous}, j_{\max}\Delta t\right)$$

This produces an approximately S-curve speed profile rather than abrupt RC changes.

Better solution: receding-horizon MPC

For higher speed, use Model Predictive Control with a 1–2 second horizon. Minimize:

$$J = \sum_{k=0}^{N} \left[w_p \|\mathbf{p}_k - \mathbf{p}_{target,k}\|^2 + w_v \|\mathbf{v}_k - \mathbf{v}_{target,k}\|^2 + w_u \|\mathbf{a}_k\|^2 + w_j \|\mathbf{a}_k - \mathbf{a}_{k-1}\right|$$

The image-error term is important: it prevents the fastest geometric trajectory from causing the target to leave the camera FOV.

---

6. Keep the target inside the image

Normalize the pixel error:

$$e_x = \frac{u - c_x}{f_x}$$

$$e_y = \frac{v - c_y}{f_y}$$

Use horizontal error primarily for yaw:

$$\dot{\psi}_{cmd} = k_{\psi} e_x$$

Limit yaw acceleration and yaw rate:

$$|\dot{\psi}_{cmd}| \leq \dot{\psi}_{max}$$

Use vertical image error as part of pitch or vertical-position guidance, but do not use it alone to infer altitude.

At high speed, define a central operating region—for example, the middle 50–60% of the image:

- Inside the central region: allow acceleration.
- Approaching the image boundary: reduce speed.
- Outside the permitted region: stop forward acceleration and prioritize recentering.
- Tracker lost: brake and enter hover or controlled climb.

---

7. Convert trajectory commands to RC commands

The preferred interface is to command attitude, body rates or velocity through a supported flight-controller mode. Sending raw RC values is possible but less precise.

Assuming you generate desired body-frame acceleration:

$$\mathbf{a}_b = \begin{bmatrix} a_x & a_y & a_z \end{bmatrix}^T$$

For moderate angles, an approximate mapping is:

$$\theta_{desired} \approx \tan^{-1}\left(\frac{a_x}{g}\right)$$

$$\phi_{desired} \approx -\tan^{-1}\left(\frac{a_y}{g}\right)$$

where:

- θ: pitch
- ϕ: roll

Convert angles into normalized joystick values:

$$pitch_{rc} = \text{clip}\left(\frac{\theta_{desired}}{\theta_{\max}}, -1, 1\right)$$

$$roll_{rc} = \text{clip}\left(\frac{\phi_{desired}}{\phi_{\max}}, -1, 1\right)$$

Yaw:

$$yaw_{rc} = \text{clip}\left(\frac{\dot{\psi}_{cmd}}{\dot{\psi}_{max}}, -1, 1\right)$$

Throttle needs calibration. A simple approximation is:

$$throttle_{rc} = throttle_{hover} + k_z a_z$$

This should include tilt compensation:

$$throttle_{tilt} \approx \frac{throttle_{hover}}{\cos\phi \cos\theta}$$

For an acro-mode FPV drone, joystick inputs command body rates rather than angles. In that case, add an attitude/rate controller between desired acceleration and the RC commands. Do not directly map position error to stick deflection.

---

8. Example guidance loop

Python Run

```python
import math
import numpy as np


def clip_norm(vector: np.ndarray, max_norm: float) -> np.ndarray:
    norm = np.linalg.norm(vector)
    if norm <= max_norm or norm < 1e-8:
        return vector
    return vector * (max_norm / norm)


def compute_guidance(
    drone_position: np.ndarray,
    drone_velocity: np.ndarray,
    target_position: np.ndarray,
    target_velocity: np.ndarray,
    previous_acceleration: np.ndarray,
    dt: float,
    *,
    maximum_speed: float,
    maximum_acceleration: float,
    braking_acceleration: float,
    maximum_jerk: float,
    safe_distance: float,
    velocity_time_constant: float = 0.5,
) -> tuple[np.ndarray, np.ndarray]:
    relative_position = target_position - drone_position
    distance = float(np.linalg.norm(relative_position))

    if distance < 1e-6:
        return np.zeros(3), np.zeros(3)

    line_of_sight = relative_position / distance

    usable_distance = max(distance - safe_distance, 0.0)
    stopping_limited_speed = math.sqrt(
        2.0 * braking_acceleration * usable_distance
    )

    commanded_speed = min(maximum_speed, stopping_limited_speed)

    desired_velocity = (
        target_velocity + commanded_speed * line_of_sight
    )

    raw_acceleration = (
        desired_velocity - drone_velocity
    ) / velocity_time_constant

    limited_acceleration = clip_norm(
        raw_acceleration,
```

```
        maximum_acceleration,
    )

    acceleration_change = limited_acceleration - previous_acceleration
    acceleration_change = clip_norm(
        acceleration_change,
        maximum_jerk * dt,
    )

    commanded_acceleration = (
        previous_acceleration + acceleration_change
    )

    return desired_velocity, commanded_acceleration
```

In the real system, also reduce `commanded_speed` according to:

Python Run

```
commanded_speed = min(
    maximum_speed,
    stopping_limited_speed,
    tracking_limited_speed,
    uncertainty_limited_speed,
)
```

Practical recommendation for your FPV system

For the first implementation:

1. Calibrate camera intrinsics and camera-to-drone rotation.
2. Obtain drone attitude and velocity from the flight controller.
3. Use VIO, GPS or optical-flow odometry for camera translation.
4. Track the target center and bounding box.
5. Perform a short sideways maneuver to triangulate range.
6. Maintain a target-position EKF.
7. Use braking-distance speed limiting.
8. Apply acceleration and jerk limits.
9. Keep yaw pointed toward the target.
10. Send smooth rate or attitude commands to Betaflight.
11. Approach to a stand-off radius rather than commanding collision with the target.

For genuinely maximum-speed operation, **a direct range sensor or stereo camera is strongly preferable**. Monocular bearing-only tracking can guide the drone toward a target, but uncertainty in distance makes maximum-speed terminal braking unreliable.

Test the complete controller in SITL with geofencing, manual takeover, target-loss braking and conservative acceleration limits before real flight.

Sources

my motivation is to keep the target at camera center i understand when i command pitch the altitude drop let take the simple scenario the we need to create straight descend path between the drone and the target

Worked for 55s

For the simplified case, the desired path is the **line of sight from the drone to the target**.

When the target is exactly at the camera center, the camera optical axis points along this line. Therefore, you can command the drone's velocity along the camera optical axis:

$$\mathbf{v}_{des} = V_{cmd}\hat{\mathbf{r}}_{LOS}$$

You do **not** need the distance to define the direction of the straight path. You only need distance later for braking and stopping.

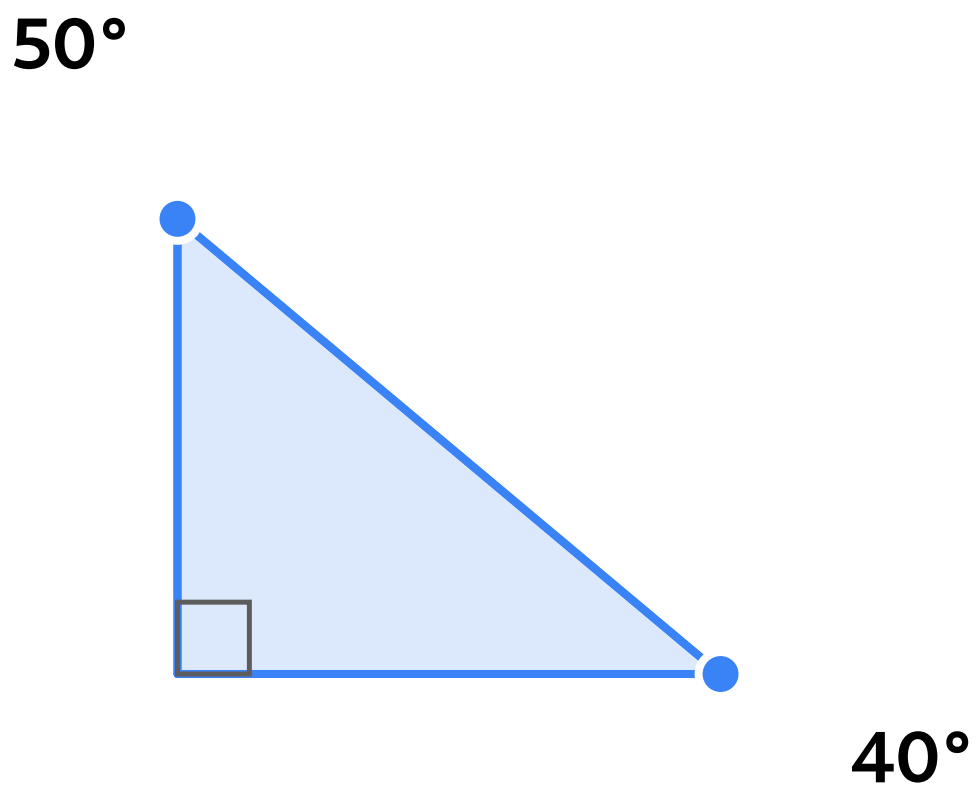
Geometry of the descent

Let:

- H: drone height above the target
- R: horizontal distance to the target
- D: direct distance to the target
- γ: descent or flight-path angle

$$D = \sqrt{R^2 + H^2}$$

$$\gamma = \tan^{-1}\left(\frac{H}{R}\right)$$



Angles 90°, 40°, and 50° include one fixed right angle

For a commanded total speed V:

$$V_{horizontal} = V\cos(\gamma)$$

$$V_{down} = V\sin(\gamma)$$

For example, with a 20° descent angle and $V = 10\ \mathrm{m/s}$:

$$V_{horizontal} = 10\cos(20^\circ) = 9.4\ \mathrm{m/s}$$

$$V_{down} = 10\sin(20^\circ) = 3.42\ \mathrm{m/s}$$

So the controller should simultaneously command approximately:

- $9.4\ \mathrm{m/s}$ forward
- $3.42\ \mathrm{m/s}$ downward

Use the camera ray directly

For a calibrated camera, the target center pixel (u, v) produces the camera-frame ray:

$$\mathbf{r}_c = \frac{K^{-1}\begin{bmatrix} u \\ v \\ 1 \end{bmatrix}}{\left\| K^{-1}\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \right\|}$$

When the target is exactly at the principal point:

$$u = c_x, \qquad v = c_y$$

then approximately:

$$\mathbf{r}_c = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Transform this ray into the navigation frame:

$$\hat{\mathbf{r}}_{nav} = R_{nb} R_{bc} \mathbf{r}_c$$

where:

- R_{bc}: camera mounting rotation relative to the drone
- R_{nb}: drone attitude from the IMU
- $\hat{\mathbf{r}}_{nav}$: direction from drone to target

Then:

$$\mathbf{v}_{des} = V_{cmd} \hat{\mathbf{r}}_{nav}$$

This automatically gives the correct forward, sideways and downward velocity components.

Why pitching causes altitude loss

When the drone is level, all thrust contributes vertically:

$$T_z = T$$

When the drone pitches by angle θ:

$$T_z = T \cos(\theta)$$

If you keep the same throttle, $T \approx mg$, the downward acceleration becomes approximately:

$$a_{down} = g - g\cos(\theta)$$

For example:

Pitch	Vertical thrust remaining	Downward acceleration
10°	98.5%	$0.15\ \mathrm{m/s^2}$
20°	94.0%	$0.59\ \mathrm{m/s^2}$
30°	86.6%	$1.31\ \mathrm{m/s^2}$
45°	70.7%	$2.87\ \mathrm{m/s^2}$

This natural altitude drop is not enough to accurately follow the desired straight line. You should control the vertical velocity explicitly rather than relying only on the loss of vertical thrust.

Coordinated pitch and throttle

Suppose the desired downward acceleration is a_{down}. The required total thrust is approximately:

$$T_{cmd} = \frac{m(g - a_{down})}{\cos(\theta)}$$

There are three cases.

Accelerating downward

$$a_{down} > 0$$

Reduce the vertical thrust component:

$$T\cos(\theta) < mg$$

Constant descent speed

$$a_{down} = 0$$

Ignoring vertical drag:

$$T_{cmd} = \frac{mg}{\cos(\theta)}$$

This may appear surprising: during a constant-speed descent, the drone still needs approximately enough vertical thrust to balance gravity. Lower thrust is needed primarily while accelerating into the descent.

Braking the descent

To reduce downward velocity, command upward acceleration:

$$T\cos(\theta) > mg$$

Simple straight-line guidance controller

Use NED coordinates, where positive z means downward.

Python Run

```python
import numpy as np


def clip_norm(vector: np.ndarray, maximum: float) -> np.ndarray:
    norm = np.linalg.norm(vector)

    if norm < 1e-8 or norm <= maximum:
        return vector

    return vector * maximum / norm


def straight_line_guidance(
    los_navigation: np.ndarray,
    current_velocity: np.ndarray,
    previous_acceleration: np.ndarray,
    commanded_speed: float,
    dt: float,
    velocity_gain: float = 2.0,
    maximum_acceleration: float = 4.0,
    maximum_jerk: float = 8.0,
) -> tuple[np.ndarray, np.ndarray]:
    """
    NED frame:
      x: north/forward
      y: east/right
      z: down
    """

    los_navigation = los_navigation / np.linalg.norm(los_navigation)

    # Desired velocity directly toward the target.
    desired_velocity = commanded_speed * los_navigation

    # Velocity tracking controller.
    raw_acceleration = velocity_gain * (
        desired_velocity - current_velocity
    )

    limited_acceleration = clip_norm(
        raw_acceleration,
        maximum_acceleration,
    )

    # Jerk limiting for smooth RC commands.
    acceleration_change = (
        limited_acceleration - previous_acceleration
    )

    acceleration_change = clip_norm(
        acceleration_change,
```

```
        maximum_jerk * dt,
    )

    commanded_acceleration = (
        previous_acceleration + acceleration_change
    )

    return desired_velocity, commanded_acceleration
```

If the LOS direction is:

Python Run

```
los = np.array([0.90, 0.0, 0.435])
```

then the target is forward and downward. At $10\ \mathrm{m/s}$:

Python Run

```
desired_velocity = np.array([
    9.0,    # forward
    0.0,    # sideways
    4.35,   # downward
])
```

Keeping the target centered

Use the target pixel error:

$$e_x = \frac{u - c_x}{f_x}$$

$$e_y = \frac{v - c_y}{f_y}$$

Horizontal error controls yaw:

$$\dot{\psi}_{cmd} = K_{yaw} e_x$$

Vertical image error should correct the flight-path angle:

$$V_{down,cmd} = V_{down,LOS} + K_v e_y$$

The sign depends on the camera coordinate convention. Conceptually:

- Target above image center: reduce descent or pitch upward.
- Target below image center: increase descent or pitch downward.

A practical speed limiter is:

$$V_{cmd} = V_{max} \cdot \max\left(0, 1 - k_e\sqrt{e_x^2 + e_y^2}\right)$$

This means:

- Target near image center → accelerate.
- Target moves away from center → reduce speed.
- Target near image edge → stop accelerating.
- Tracker lost → brake or hover.

Important fixed-camera limitation

For a fixed FPV camera, two objectives are coupled:

1. Point the camera at the target.
2. Tilt the drone to generate acceleration.

The drone attitude required to accelerate is not always the same as the camera attitude required to center the target. Therefore, perfect centering during aggressive acceleration may be impossible with a fixed camera.

The options are:

- Add a gimbal.
- Mount the camera at the expected approach angle.
- Allow a small image-center error during acceleration.
- Use an MPC controller that balances trajectory tracking and image centering.

For example, define an allowed center zone:

$$|e_x| < 0.15, \qquad |e_y| < 0.15$$

Instead of requiring exactly zero error.

Recommended implementation

For the first version:

1. Obtain the target center from the tracker.
2. Convert the center pixel to a 3D camera ray.
3. Rotate that ray into the navigation frame using the drone attitude.
4. Set desired velocity parallel to the ray.
5. Smoothly increase the desired speed.
6. Use a velocity controller to generate acceleration commands.
7. Convert acceleration into pitch, roll and thrust commands.
8. Use pixel error to correct yaw and vertical velocity.
9. Reduce speed when the target moves away from the image center.

10. Because range is unknown, do not attempt terminal braking based only on the camera bearing.

The core guidance law is simply:

$$\boxed{\mathbf{v}_{des} = V_{cmd}\hat{\mathbf{r}}_{LOS}}$$

This produces the straight descending trajectory from the drone toward the target while keeping the velocity direction aligned with the centered camera target.

Today 3:02 PM

can i save this chat as pdf

Yes. On Ubuntu/Firefox:

1. Press **Ctrl + P** while this chat is open.
2. Under **Destination**, select **Print to File** or **Save to PDF**.
3. Choose the filename and folder.
4. Enable **Background graphics** if you want the formatting preserved.
5. Click **Print** or **Save**.

This saves the currently displayed conversation as a PDF. Scroll through the full chat first so older messages are loaded before printing.

For a complete archive of all conversations, use **Profile → Settings → Data Controls → Export Data**. OpenAI sends a downloadable ZIP containing your chat history; this is an account-data export rather than a ready-made PDF. help.openai.com

Sources